

to support IoT switches with Alexa and Google Assistant compatibility. In this system, the same RainMaker platform is extended by integrating current and voltage sensing with a smart billing mechanism, resulting in a comprehensive, production-ready energy-management solution.

Fig. 1 shows the ESP RainMaker app interface with an integrated smart home dashboard and real-time energy monitoring. The left screen displays multiple smart switches and an energy-monitor device, while the right screen presents live electrical parameters such as voltage, current, power, energy consumption, and estimated electricity bill. Fig. 2 shows the connected device with the RainMaker app displaying power and energy consumption.

To build the system, the required components are listed in the Bill of Materials table.

Calibration circuit

Fig. 3 shows the calibration circuit. The relay controls the appliance on or off. In this design, a single relay is used. The ACS current sensor measures AC current and frequency, while the ZMPT101B module performs voltage measurement.

Additional relays can be added to control more appliances using available I/O pins. Since the sensor operates at 5V logic and the controller uses 3.3V logic, a voltage divider is required to reduce the signal level. A 2k Ω and 10k Ω resistor divider is used for this purpose.

Calibration of sensor

First, the ZMPT101B voltage sensor must be calibrated. Connect the sensor to the IndusBoard Coin (refer to Fig. 3) and upload the calibration code.

```
//-----
void setup() {
  Serial.begin(115200);
}

void loop() {
  int raw = analogRead(3); // Your
```

```
8 // Uses manual sampling for measurements
9 // Adjust constants as needed (e.g., COST_PER_UNIT, thresholds)
10
11 #include "RMaker.h"
12 #include "WiFi.h"
13 #include "WiFiProv.h"
14
15 const char *service_name = "RainMaker_BLE";
16 const char *pop = "12345678";
17
18 // Sensor Pins and Constants
19 const int SENSOR_PIN = 4; // ACS712
20 const int V_SENSOR_PIN = 5; // ZMPT101B
21 const int RELAY1_PIN = 6;
22 const int RELAY2_PIN = 33;
23 const float SENSOR_VCC = 5.0;
24 const float ADC_VCC = 3.3;
25 const float SENSITIVITY = 0.185; // ACS712 5A
26 const float DIVIDER_RATIO = 22.0 / (10.0 + 22.0); // 10k top + 22k bottom
27 const float V_DIVIDER_RATIO = 1.0;
28 const float V_SENSITIVITY = 500.0f;
29 const float NOISE_THRESHOLD_MA = 150.0;
30 const float COST_PER_UNIT = 0.15; // $/kWh
31
32 // Protection Thresholds
33 const float OVER_VOLTAGE = 250.0;
34 const float UNDER_VOLTAGE = 180.0;
35 const float OVER_CURRENT = 5.0;
36
37 // Reporting Interval (ms)
38 const unsigned long REPORT_INTERVAL = 3600000UL; // 1 hour
39
```

Fig. 7: RainMaker code snippet showing calibration parameters and sensor pin configuration

```
pinSerial.println(raw);
delay(1); // Fast for Serial Plotter
}
//-----
```

Next, open the serial plotter and rotate the potentiometer using a screwdriver, as shown in Fig. 4, until a clean sine waveform appears. Fig. 5 shows the sine waveform observed on the serial monitor during adjustment.

Voltmeter sensor testing code

After a clean sine wave is obtained, the full waveform becomes visible on the plotter without clipping. The voltage can then be measured using the ZMPT101B example code. If a multimeter is available, the sensor reading should be compared with the multimeter value. Any difference can be corrected through further calibration by adjusting the sensitivity value in the code.

```
#include <ZMPT101B.h>
#define SENSITIVITY 500.0f
```



Fig. 8: Device QR code generated for provisioning

```
// ZMPT101B sensor output connected to
analog pin A0
// and the voltage source frequency is
50 Hz.
ZMPT101B voltageSensor(3, 50.0);
void setup() {
  Serial.begin(115200);
  // Change the sensitivity value based on
  value you got from the calibrate
  // example.
  voltageSensor.setSensitivity(SENSITIVITY);
}
```